

SELECTED PROJECT / FINTECH INFRASTRUCTURE

Every movement. Accounted for.

FinTech Payment Ledger

A self-hosted wallet and payment system that makes money movement explainable, repeatable, and auditable.

An independent engineering project covering the accounting core, REST API, React operations console, provider-event simulation, and repeatable deployment.

12

Release tests passed

4

CI jobs passed

2

Image architectures

1

Startup command

Verified v0.3.0 release baseline; source links on page 3.

TRANSFER / GBP 25.00

ILLUSTRATIVE POSTING

Sender wallet	-2,500 GBP
Recipient wallet	+2,500 GBP
Sum per currency	0

Amounts shown in signed integer minor units. No money is created by this transfer.

TECHNOLOGY

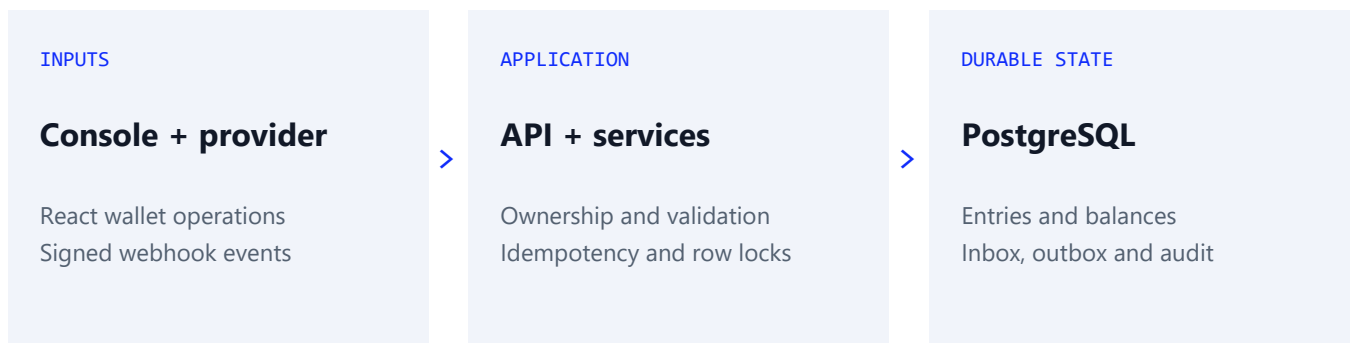
Java 21 / Spring Boot / PostgreSQL / Spring JDBC / Flyway
React / TypeScript / Docker Compose / GitHub Actions

[Public repository: github.com/QihuiPan/fintech-payment-ledger](https://github.com/QihuiPan/fintech-payment-ledger)

01 / DESIGN

Correctness at the posting boundary.

Repeated requests, concurrent spending, and reversals share one set of accounting rules. Each posting is enclosed in a database transaction.



One atomic commit: ledger entries + balance projections + outbox event

01 Use integers for money

Signed minor units avoid binary floating-point amounts. FX rates use fixed precision; every currency leg balances independently.

02 Check funds after acquiring account locks

Stable account ordering prevents lock inversion. The available-balance check runs inside the posting transaction.

03 Make retries and corrections explicit

A stored request fingerprint identifies a valid replay. Reversals append opposite entries and preserve the original record.

[Architecture and deposit sequence](#)

[PostgreSQL append-only and deferred balancing guards](#)

Evidence you can inspect.

The v0.3.0 baseline passed 12 automated tests and all four CI jobs. The following cases are linked to the exact released source.

Retry semantics

The same deposit and key return the original transaction ID. Changing the amount while reusing the key is rejected.

[Payment flow integration test](#)

Concurrent spending

Two GBP 8.00 transfers compete for a GBP 10.00 balance. Exactly one succeeds and GBP 2.00 remains.

[Concurrent spend integration test](#)

Accounting rules

1,000 randomized balanced examples are accepted. An unbalanced currency leg is rejected; FX balances separately in each currency.

[Ledger math tests](#)

HTTP access controls

Ownership checks and administrator overrides are covered, along with metrics permissions and a disabled default H2 console.

[HTTP security integration test](#)

BUILD AND DELIVERY

Backend tests, frontend build, PostgreSQL migration/readiness, and image build passed. The release workflow published AMD64 and ARM64 images.

[CI run](#)

[Container release run](#)

[v0.3.0 release](#)

Evidence scope: core integration tests use H2; the PostgreSQL job checks startup and migrations. These results are not throughput, availability, or certification claims. Baseline commit: 7e5b2f9. Portfolio reviewed: 11 September 2026.

A project you can run.

Docker Compose packages the English console, API, PostgreSQL, migrations, health checks, and persistent storage into a local evaluation environment.

QUICK START / DOCKER + COMPOSE V2

```
git clone https://github.com/QihuiPan/fintech-payment-ledger.git
cd fintech-payment-ledger
docker compose up --build --wait
```

A short evaluation path

- 01 Open localhost:8080 and select Create demo workspace.
- 02 Inspect the seeded GBP 100.00 deposit and its balanced entries.
- 03 Transfer funds, execute an FX quote, and reverse a transaction.
- 04 Review the running statement and the administrator invariant check.

Latest extension: trace-linked telemetry

After v0.3.0, the main branch added optional OpenTelemetry export, HTTP and worker signals, correlated request logs, and Prometheus exemplars. These additions connect operational symptoms with request traces.

[Inspect the observability integration at c0edf44](#)

Next engineering steps

The current application is a runnable reference with a simulated provider and a configurable in-memory identity directory. Further work includes external identity, a durable outbox relay, coordinated multi-worker processing, and measured load and recovery testing before real-money service operation.

[Deployment guide](#)

[Threat model and scope boundaries](#)